ADR 0012 — Model Registry

Status: Proposed (Marv to confirm after review + open items resolved with Ronin) **Date**: 2026-08-21 **Author**: Marv **Related**: ADR 0004, ADR 0011, docs/METHODOLOGY.md §8, docs/BACKEND_FLOW.md §6, RONIN_SYNC.txt B-tier B2/B3/B4

Context

Phase 2 introduces multiple served model versions:

- Base OpenAI models (`gpt-4o-mini`, potentially `gpt-4o`) — no artifacts stored locally
- Base Qwen 2.5 1.5B checkpoint (per ADR 0004) — weights on disk
- Fine-tuned Qwen + LoRA adapters (`qwen-1.5b-baymax-v1`, `-v2`, ...) — Ronin produces per SFT run

The agent (via `MLXBackend` per ADR 0011) needs a stable answer to: *"given `model_id = X`, where do I find its weights and what facts do I need to know about it?"*

Naive alternative: hardcode paths and metadata in backend code. Problems:

- Every new fine-tune = backend code change (slows Ronin's training iteration)
- Backend's hardcoded facts and reality can drift silently
- No audit trail for "which model produced which eval result"
- No enforcement of "immutable = new id per retrain" (Ronin could accidentally overwrite v1)

A registry solves all four.

Constraints:

- Zero new infrastructure (matches ADR 0006 / 0007 posture)
- Ronin writes (training pipeline output); Marv reads (backend load)
- Immutability required for eval reproducibility (per RONIN_SYNC B3): a `model_id` labeled result must be reproducible forever
- Must integrate with backend construction per ADR 0011 §7
- Ronin's proposed manifest schema (2026-07-29 chat) serves as the starting point; three open issues need resolving

---

Decision

1. Registry format and location

Manifest-based (Option A). Single `models/models.json` file at repo root. JSON array of model entries; backend reads at construction time. Model artifacts live at `models/<model_id>/` alongside.

Directory layout:

```
models/
├── models.json                          # the manifest
├── qwen-1.5b-baymax-v1/
│   ├── adapter_model.safetensors        # LoRA adapter weights
│   ├── adapter_config.json              # PEFT config
│   └── training_config.json             # hyperparameters snapshot
├── qwen-1.5b-baymax-v2/
│   └── ...
└── gpt-4o-mini/                         # for API-backed models, entry exists but no local
weights
    └── (empty; entry in manifest only)
```

2. Manifest schema

Each entry is a JSON object with the following fields:

Required (backend refuses to load if missing):

Field	Type	Description
`id`	string	Unique model identifier. Convention: `<base>-<variant>-<version>` e.g. `qwen-1.5b-baymax-v1`. Dashes not underscores.
`base`	string	Base model identifier. For adapters, the base checkpoint they stack on (e.g., `qwen-1.5b`). For base models, same as `id`.
`adapter_path`	string | null	Filesystem path to adapter weights, relative to repo root. `null` for base models with no adapter.
`trained_at`	ISO 8601 datetime	When the training run completed. `null` for base models pulled from HF Hub.
`training_config_hash`	string (sha256)	Hash of the training config JSON — reproducibility invariant. `null` for base models.

Optional (limited to Ronin's original schema — dropped speculative fields that add no functional value for the current scope):

Field	Type	Description
`git_commit`	string (sha)	Commit SHA of the codebase state at training time. From Ronin's original schema; reproducibility trail beyond just training config.

Field	Type	Description
`checkpoint_format`	string	Format identifier — e.g., `"peft_lora"`, `"safetensors"`, `"mlx"`. From Ronin's original schema. Helps backend pick the right loader.
`mlx_compatible`	bool	Whether the checkpoint is directly loadable by MLX (post-conversion). Defaults to `false` if absent.
`mlx_artifact_path`	string | null	Path to MLX-converted artifact, if separate from `adapter_path`.
`training.cost_usd`	float	If cloud-rented GPU.

Explicitly NOT in the registry (deferred as feature creep for a codebase not under review):

- `notes`, `warnings`, `status`, `hyperparameters`, `data_split_seed`, `epochs_completed`, `training.gpu_hours`, `training.wall_clock_seconds`, `max_context`, `cost_model`, `tokenizer_path` — all droppable. Hyperparameters recoverable from `training_config_hash`; runtime facts (max_context, cost_model, tokenizer_path) come from HF base model config or hardcoded per backend class per ADR 0011 §7 (see amended §7 there). If any of these become genuinely needed later, un-defer at that point.

Naming consistency note: Ronin's actual training config file (per `configs/sft/qwen2_5_1_5b_lora_v1.json` on his `feat/phase1-scenario-loader` branch) uses `config_hash` for what this ADR calls `training_config_hash`. Same concept, different name. Sync with Ronin to align — either the registry adopts `config_hash` or his training config emits both keys.

Explicitly NOT in the registry:

- **Benchmark scores** — live in `results/*.json` per model, NOT here. Registry is identity; results are metrics. Duplication would cause staleness. To answer "which model has the best score?" join `models.json` with `results/` at read time (~20 lines of Python).
- **Training data pointers** — belong in the results file that references this model, not in the registry entry itself. The registry is about the model artifact; the training data was an input to it, referenced from the results output.

3. Immutability + monotonic versioning

- Model IDs are **immutable**. A given `id` NEVER has its metadata rewritten after registration.
- Retraining produces a NEW id: `qwen-1.5b-baymax-v1` → `qwen-1.5b-baymax-v2` → `qwen-1.5b-baymax-v3`.
- Old entries stay in the manifest indefinitely. Old results labeled with old ids stay reproducible forever.
- If a model version is deprecated (superseded, known-broken), mark it via `status: superseded` — do NOT delete the entry.

Naming convention: dashes (`-`), lowercase, monotonic integer version suffix. Adapter differentiator uses the project tag (`baymax`), not the training method (`lora`). Examples:

- ✅ `qwen-1.5b-baymax-v1`
- ✅ `qwen-1.5b-baymax-v2`
- ✅ `gpt-4o-mini`

- ❌ `qwen25_1_5b_lora_v1_001` (Ronin's initial proposal — see §Open items)
- ❌ `qwen-1.5b-baymax-v1a` (letter suffix — deviates from monotonic integer convention)

4. Registry integration with backend

Per ADR 0011 §7, backends read from the registry at construction time:

```
class MLXBackend(InferenceBackend):
    def __init__(self, model_id: str) -> None:
        entry = load_registry_entry(model_id)  # raises ModelNotFound / IncompleteEntry
        self.model_id = entry["id"]
        self.base = entry["base"]
        self.adapter_path = entry["adapter_path"]
        self.max_context = entry.get("max_context")
        self.cost_model = entry.get("cost_model", "amortized")
        # ... load weights from self.adapter_path, etc.
```

Registry lookup is one file read per process lifetime (backend caches results). Fails loud if model_id not found or required fields missing.

5. Write path (Ronin's training pipeline)

Ronin's SFT training pipeline is responsible for:

1. Writing adapter weights to `models/<new_id>/`
2. Emitting `training_config.json` alongside
3. Computing `training_config_hash = sha256(training_config.json bytes)`
4. Appending a new entry to `models/models.json` with all required fields
5. Committing both the weights + manifest patch in the same PR

Per METHODOLOGY §3, Ronin owns registry writes. Marv owns registry reads (via backend). Manifest updates are a coordination point; Ronin's PR titles should tag `[registry-update]` for visibility.

Discipline check per RONIN_SYNC A1 pre-meeting note: since Ronin uses AI code assist, prompt his AI to include manifest patching as part of the training pipeline template. Automated update via the SFT pipeline is more reliable than a manual "remember to update the JSON" step.

Drift mitigation (future): a CI check that fails if `set(dirs in models/)` ≠ `set(ids in models.json)`. Not v1 required; add later if drift becomes a real problem.

6. Loading and validation

Registry loader lives in `src/baymax/core/registry.py`:

```
def load_registry_entry(model_id: str, path: Path = Path("models/models.json")) -> dict:
    """Loads and validates the entry for a given model_id."""
    manifest = json.loads(path.read_text())
    for entry in manifest["models"]:
        if entry["id"] == model_id:
            _validate_required_fields(entry)
```

```
            return entry
    raise ModelNotFound(model_id)
```

Validation checks all required fields per §2 are present. Raises `IncompleteEntry` with the missing field name on failure.

JSON Schema validation: full schema at `docs/schemas/model_registry.schema.json` (to write). CI validates `models.json` against schema on every PR.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months.]

- **WHY manifest file (Option A) rather than directory scan (Option B) or hybrid (Option C)**: metadata queryability. A manifest can be inspected in one file open; a directory scan requires opening N `metadata.json` files. For a small registry (~10 models over the project's life), manifest cost is negligible and manifest benefits (single source of truth, easy diff review, one place to inspect) dominate.
- **WHY the manifest drift risk (Ronin adds a folder but forgets to update the manifest) is acceptable**: mitigation via Ronin's AI-assisted training pipeline (per RONIN_SYNC A1). Automated update is more reliable than manual discipline. CI check as backstop if drift becomes a real problem — cheap safety net addable later.
- **WHY immutable + monotonic increment (not overwrite, not letter suffixes)**: eval reproducibility depends on it. If `v1` results in the results file mean anything, `v1` weights must be forever recoverable. Overwriting v1 breaks every prior labeled result. Letter suffixes (`-v1a`, `-v1b`) invite ambiguity about ordering. Monotonic integers = unambiguous, machine-comparable, minimally cognitive-overhead.
- **WHY only 5 required fields (id, base, adapter_path, trained_at, training_config_hash)**: minimum needed to LOAD the model + trace it back to reproducibility. Everything else is optional; require the minimum, encourage the rest. `training_config_hash` is required specifically because config file paths can drift silently — the hash is the reproducibility invariant.
- **WHY benchmark_score is EXPLICITLY NOT in the registry**: two sources of truth = drift. If the eval is re-run (scorer bug fixed, expanded scenarios, tier stratification), the benchmark score in the registry entry becomes wrong. Results live in `results/*.json`; registry entries reference them via path. Joining registry + results at read time is trivial (~20 lines of Python); duplicating benchmark scores in the registry means one becomes stale and reviewers don't know which to trust.
- **WHY separate manifest + `models/<id>/` folder structure** (not everything in the manifest, not everything in the folder): manifest is the fast queryable index (which models exist, what are their facts). Folder is the storage (weights are gigabytes, not manifest-appropriate). Standard pattern in ML tooling (HuggingFace does essentially this).
- **WHY Ronin writes, Marv reads**: matches author-split discipline (METHODOLOGY §3). Ronin's training pipeline produces the artifact; the artifact is the write. Marv's backend consumes it; consumption is the read. Cross-role writes would create coordination overhead and blur ownership.
- **WHY JSON Schema validation in CI**: catches malformed entries before they reach the backend at startup. Backend startup validation is the safety net; CI validation is the fast-fail. Same principle as any typed contract — validate at the boundary.

- **WHY `status` field is optional** (defaults to `completed`): overwhelming majority of entries will be completed models. Making status required forces every entry to include a field that's usually the same value. Optional-with-default = pragmatic.
- **WHY defer HF Hub as the registry backend to v2**: v1 is local-machine only (Ronin's training cloud + Marv's Mac). No multi-user, no cross-machine sharing, no discoverability needs. Filesystem is enough. When v2 needs artifact sharing (external collaborators, publishing to HuggingFace), the manifest schema migrates cleanly to HF Hub's model card format (which is a superset).
- **WHY register API-backed models (gpt-4o-mini) in the registry too, despite no local artifacts**: uniformity. Backend construction always goes through `load_registry_entry(model_id)` regardless of backend type. If OpenAI models aren't registered, `OpenAIBackend` needs a special-case path — violates uniformity, invites drift. Register them with `adapter_path: null` and empty folder; backend code doesn't branch.

Alternatives considered

Alternative A — Directory-scan discovery (no manifest)

Registry `list_models()` = `[p.name for p in Path("models/").iterdir() if p.is_dir()]`. Metadata lives in per-folder `metadata.json`.

Why rejected: no global queryability without opening N files. Adding a new model doesn't require any manifest update, which is a positive, BUT the query cost + lack of one-file-diff review is a negative. For a small registry (~10 models over project life), manifest cost is negligible.

Alternative B — Hybrid (directory scan for discovery + per-folder metadata.json)

HuggingFace-style. Directory is the truth for existence; per-folder metadata.json for facts.

Why rejected: two-layer indirection (open manifest → open per-folder file → read fields). More complex than a single manifest for a project this small. Would be the right answer at HF Hub scale; not at BAYMAX scale.

Alternative C — Database-backed registry (SQLite table)

Store registry entries as rows in a SQLite table (potentially the same DB as the state store per ADR 0007).

Why rejected: overkill for a read-mostly, small (~10-entry) dataset. Adds a schema migration story, ORM decision, backup complexity. JSON file is git-diffable, human-readable, trivially auditable. If we ever grow past ~1000 model versions, revisit.

Alternative D — Backend-internal metadata (rejected in ADR 0011 too)

Each backend hardcodes per-model knowledge; no registry file.

Why rejected: adding a new fine-tune = backend code change. Slows Ronin's iteration. Also violates single-source-of-truth (backend's hardcoded facts can drift from reality).

Alternative E — Include benchmark_score in registry

Add `benchmark_score` as a required field on each entry.

Why rejected: creates staleness risk. If eval is re-run, the registry entry's score is wrong until manually updated. Two sources of truth = one becomes stale = reviewers can't trust either. Registry is identity; results are metrics. Keep them separate; join at read time.

Alternative F — Letter-suffixed versions (v1a, v1b)

Retraining produces `qwen-1.5b-baymax-v1a`, then `v1b`, staying under the same major version.

Why rejected: ambiguous ordering (`v1c` vs `v2`?). Monotonic integers are unambiguous and machine-comparable. Also, if v1a "counts as" v1 for eval purposes, immutability is unclear — either v1 = v1a (which weights?) or v1 ≠ v1a (in which case why not just use v2?).

Consequences

Positive

- Adding a new fine-tune = registry entry + weights drop. Zero backend code change.
- Registry is auditable — one file, git-tracked, diffable per commit.
- Old model versions stay reproducible forever (immutability).
- Marv's backend code doesn't care about model-specific facts; just reads them.
- Ronin's training pipeline has a clear write target; artifact + manifest patch as one PR.
- JSON Schema validation in CI catches malformed entries before backend startup.
- Registry format easily migrates to HF Hub in v2 (superset of what HF requires).

Negative

- Ronin must remember to update the manifest on every training run (mitigated by AI-assisted pipeline automation).
- Manifest becomes a load-bearing file; corruption or malformed JSON at startup = process exit (though this is arguably positive — fails loud).
- Coupling between backend (`MLXBackend.__init__`) and registry (`load_registry_entry`) means registry changes can break backend startup. Contract via the schema mitigates.
- Over time, `models/models.json` grows unboundedly with deprecated entries. If it becomes unwieldy (say 100+ entries), add a "current" flag or move deprecated entries to `models/models.deprecated.json`.

Neutral

- API-backed models (OpenAI) register with `adapter_path: null` — same code path as adapter-backed models. Uniformity has a small cost (empty folders) for a large benefit (no backend branching).
- Registry is single-machine v1. Multi-machine sharing = migrate to HF Hub or similar in v2.

Open items (need resolving with Ronin before locking)

Per critique of Ronin's proposed manifest schema (chat, 2026-07-29):

1. Remove `evaluation` block from Ronin's proposed manifest

Ronin's proposal included:

```
"evaluation": {
  "task_success_rate": 0.0,
  "average_tool_call_accuracy": 0.0,
  ...
}
```

Per this ADR §2 and METHODOLOGY §8: benchmark scores live in `results/` only, NEVER in the registry. Sync with Ronin — remove this block.

2. Add `training_config_hash` field (required)

Ronin's proposal has `training.config_file` (path reference) but no hash. Path can drift silently; hash is the reproducibility invariant. Sync — add.

3. Clarify `run_id` vs `model_id` distinction

Ronin's `run_id: "qwen25_1_5b_lora_v1_001"` mixes model version + run number. Two possible interpretations:

- (a) `run_id` IS the model identifier; every training run produces a distinct model
- (b) `run_id` is a training-execution log identifier; `model_id` is separate; multiple runs can produce the same model version

Under interpretation (a), immutability holds but naming is weird (`_001` suffix on a "version 1" model).

Under interpretation (b), we need BOTH `run_id` AND `model_id` as separate fields.

Sync with Ronin to lock which — and adjust the naming convention (§3) to match.

4. MLX conversion plan

Ronin's proposal has `"mlx_compatible": false, "mlx_artifact_path": null` — honest about the PEFT adapter not being directly MLX-loadable. But:

- When does conversion happen? (Post-training in Ronin's pipeline? On-demand by Marv's backend? Manual?)
- Where does the MLX artifact land? (Overwrite the folder? Sibling `models/<id>_mlx/`?)
- Does conversion update the existing registry entry (violates immutability) or create a NEW entry with `id: <original>-mlx`?

Suggested resolution: new registry entry per format. `qwen-1.5b-baymax-v1` (PEFT) and `qwen-1.5b-baymax-v1-mlx` (converted). Preserves immutability. Backend selects the appropriate format via `model_id`.

Sync with Ronin to confirm.

Open questions / follow-ups

- **CI schema validation**: write `docs/schemas/model_registry.schema.json` + add GitHub Actions check.

- **CI drift check**: add script that fails if `set(models/*/)` ≠ `set(models.json ids)`. Optional; add when drift becomes a real concern.
- **Model retention policy**: no automatic deprecation of old entries in v1. When we cross ~50 entries, consider adding.
- **Registry ranking script**: `scripts/rank_models.py` that joins `models/models.json` with `results/*.json` to answer "which model performs best on which tier?" Small script; write when Ronin has enough model versions to make ranking meaningful.
- **HF Hub migration path**: v2 target. Manifest schema is designed to be a superset of HF model-card frontmatter for clean migration.

References

- ADR 0004 — Qwen 2.5 1.5B base choice
- ADR 0011 — backend that consumes this registry (§7)
- docs/METHODOLOGY.md §8 — required fields policy, benchmark_score in results/ only rule
- docs/BACKEND_FLOW.md §6 — registry lookup diagram
- RONIN_SYNC.txt B2, B3, B4 — Phase 2 locked decisions on registry
- HuggingFace Hub model cards: https://huggingface.co/docs/hub/model-cards — future-migration target format
- Ronin's proposed manifest schema (chat, 2026-07-29) — starting point for §2

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.